Paper Review • arXiv 2611.13028

GTAGym

Real AAA Game Tasks Environment for Evaluation and Reinforcement Learning

Turning GTA V main missions into auto-scored, mass-generated evaluation and RL environments

Jiajun Chen, Shicheng Fang, Jiahao Lu, Yuxin Wang † , Xipeng Qiu †

Shanghai Innovation Institute • Fudan University (OpenMOSS) • 2026-11-09

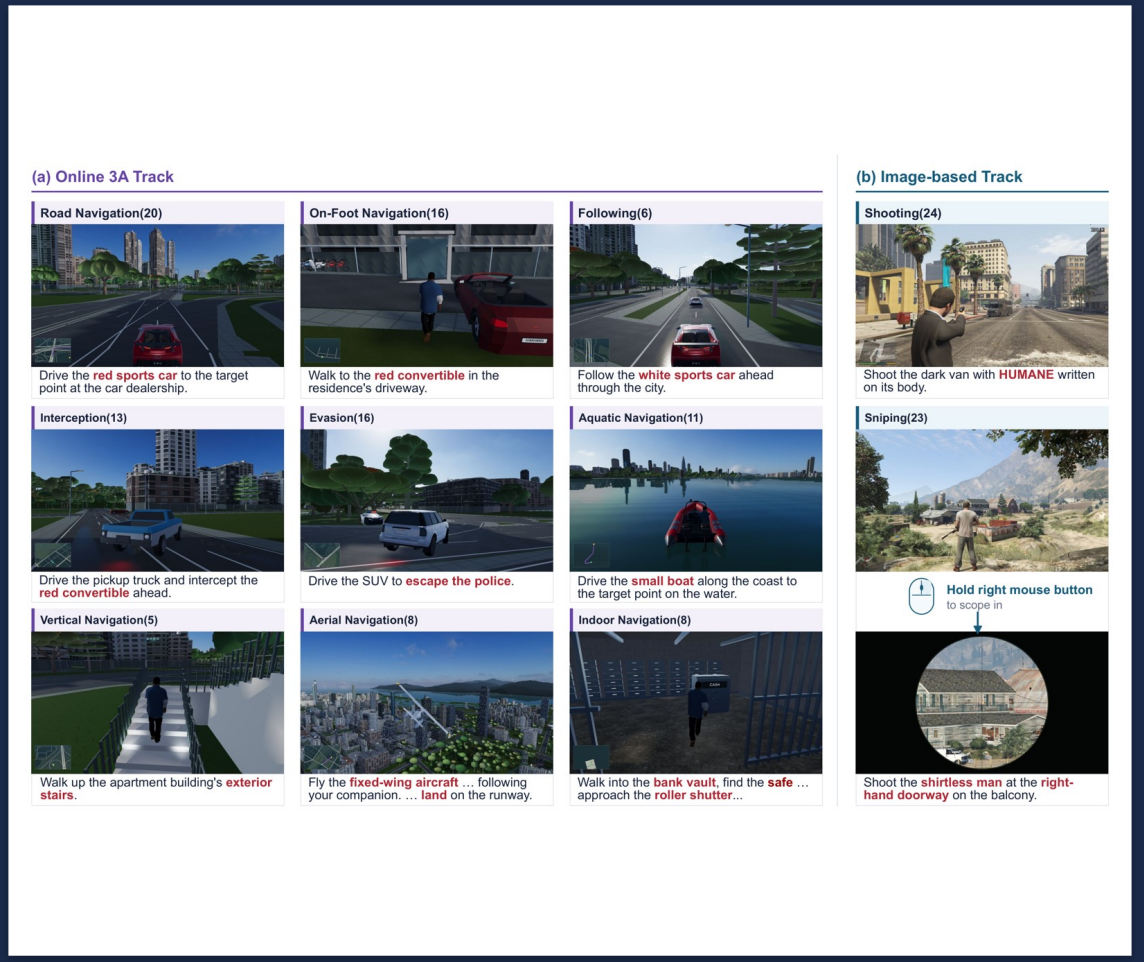


Figure: GTAVBench task suite overview (paper Figure 2)

One-Sentence Summary

GTA V mission structure, engine-state scoring; agents lag humans markedly

The authors split **75 GTA V main missions** into **150 tasks whose success/failure is judged automatically from engine state**, plus a parameterized RL environment generator. The strongest VLM agent reaches only **70.9%** (humans 89.8%), with weaknesses concentrated in **continuous closed-loop control such as flying, interception and evasion**, and accuracy plunges without the inference pause. LoRA-GRPO on just 50 generated tasks improves both in-distribution and native GTA V results.

150

tasks

103 online 3D + 47 image shooting/sniping, derived from 75 main missions

70.9%

Best model

GPT-6-Astra + Codex; humans 89.8%, gap 18.9 pt

−22.3 pt

No inference pause

GPT-6.1-Sol on 3D tasks: 62.1% → 39.8%

15→23%

Native GTA V

Qwen3.8-27B after LoRA-GRPO on 50 generated on-foot tasks

Data: paper Table 2, Section 4.3, Section 5.4

Motivation

Existing game benchmarks each miss a piece; GTAGym tries to combine them

① Verifiable tasks in sandbox / custom games

MineDojo · GameWorld · OmniGameArena

Gap: not AAA open-world tasks

② Direct play in commercial 3D games

Cradle (RDR2) · SIMA · Lumine · OctoGTA

Gap: human/OCR/VLM judging; hard to parallelize

③ Video-trajectory or offline QA benchmarks

VideoGameBench · GameVerse · GameHorizon-Bench

Gap: not online closed-loop, or scored by VLM milestones

Work	3D interaction	AAA open-world tasks	State-based scoring	Web execution	RL instance generation
Procgen	—	—	●	—	●
MineDojo	●	—	●	—	●
VideoGameBench	●	—	—	●	—
GameVerse	●	●	—	—	—
GameWorld	●	—	●	●	—
OmniGameArena	●	—	●	—	—
GameHorizon-Bench	●	—	—	—	—
GTAGym (this work)	●	●	●	●	●

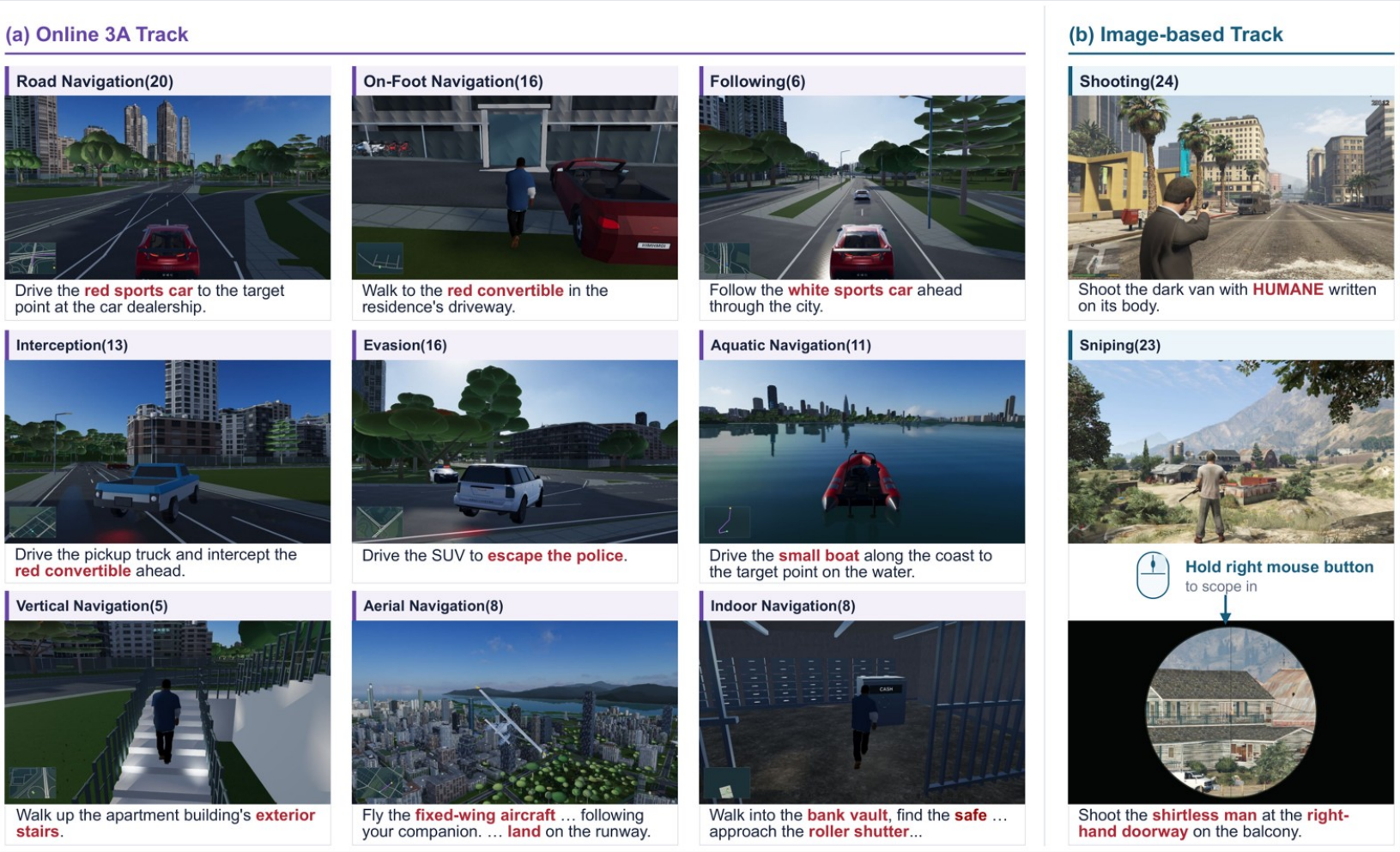
● Supported — Not reported

GTAGym is the only work in the table supporting all five

Table: paper Table 1

GTAVBench Task Suite

11 skill categories, 150 tasks covering navigation, interaction and gunplay



Navigation — **68** tasks

Road 20 • On-Foot 16 • Aquatic 11
Vertical 5 • Aerial 8 • Indoor 8

Interaction — **35** tasks

Following 6 • Interception 13
Evasion 16

Gunplay — **47** tasks

Shooting 24 • Sniping 23
(image track, real GTA V frames)

Note: online 3D scenes run in a Babylon.js city rebuilt from open-source GTA_SZ, not the GTA V engine; "real AAA" refers to task structure from main missions.

Figure: paper Figure 2; task counts from Table 2

Walkthroughs → executable predicates: tasks auto-judged by engine state

01 Decomposition

Using walkthrough videos, 75 main missions are manually split into subtasks annotated with goals, interaction order, completion constraints and NPC behavior

02 Scene Rebuild

Babylon.js 3D environment; original route traced on the source minimap, then a similar-turn-count route chosen; NPC behavior, collision slowdown set

03 Instantiation

Each subtask = initial state + instruction + turn budget + executable success / failure predicates

04 Auto-Judging

Online tasks read engine state; image tasks check annotated hit regions; binary success, no partial credit

Example judging predicates

An agent's own “done” claim does not count; the predicate must hold

Interception

Target speed ⩽ 3 m/s for 1.5 s straight; agent ahead of the target with gap < 2.5 m and own speed ⩽ 4 m/s. Motorcycle targets need 3 separate knockdowns.

Evasion

Within a 180 s limit, stay out of all scan sectors (12 m radius, 90°) for 60 consecutive s; 3 s cumulative inside a sector means capture.

Landing (Aerial)

Takeoff must be logged; land on the designated runway / helipad at ⩽ 1.5 m/s, throttle < 5%, stationary for 2 s. Crashing or leaving the airspace fails.

Sniping

The crosshair point is inverse-mapped via camera and zoom into annotated coordinates; K targets get only K bullets, and any miss or repeat hit fails.

Source: paper Section 3.2, Appendix A.2

Same tasks/actions; differ only in instruction format & context mgmt

Shared settings

- Screenshot → action closed loop, keyboard + mouse control
- Each key has a hold duration d ∈ (0, 1] s, keys may overlap; turn duration = longest key press
- Default: **game paused during model inference**
- Budget: navigation / interaction ⩽ 300 turns, gunplay ⩽ 20 turns
- Metric: micro-averaged accuracy over 150 tasks, 3 random seeds
- Human baseline: 3 players who have completed GTA V, each doing all tasks once
- 6 models: GPT-6-Astra, GPT-6.1-Sol, DeepSeek-V4.1-Flash, GLM-5.3-Flash, Kimi-K3, Qwen3.8-27B

GTAVBench

Plain GUI protocol

- Text task instruction + fixed system prompt
- Model outputs JSON actions directly; an adapter validates, then executes them
- Online tasks: history capped at 10 screenshot–reply pairs; oldest 5 dropped on overflow
- Image tasks keep the full history

GTAVBench-AGI

Codex harness

- Task goal shown only at the top-left of the first screenshot, mimicking in-game mission prompts
- Actions submitted via the game_step tool
- Codex native history + automatic context compaction
- Later tool results no longer repeat the goal

```
{"keys": [{"key": "w", "duration_seconds": 1},
          {"key": "Space", "duration_seconds": 0.1}],
 "mouse": {"move": {"dx": 120, "dy": 30}}}
```

Example action: look right 120 px, up 30 px; hold W 1 s + Space 0.1 s (Appendix A.3)

Source: paper Sections 3.3, 4.1, Appendix A.3–A.5, D

Template + param. ranges → batch RL envs with reward & done

$$\theta \sim P_T(\cdot|C) \qquad E = G(T, \theta; \xi)$$

T Template	Fixes scene, assets, interaction logic, judge and turn budget
C Constraints	User-specified numeric ranges or candidate sets
θ Parameters	Sampled within constraints and template feasibility; infeasible combos rejected, failure reported
ξ Seed	Resolved config and seed are saved, making instances reproducible

Reward & termination: task outcome is a sparse reward; success / failure ends the episode, budget exhaustion counts as truncation. 3D tasks use engine-state predicates; image tasks use hit checks.

User-constrainable sampling parameters

Start / goal	Initial heading	Route length
Number of turns	Domain (land/water/air)	Locomotion mode
NPC speed	Initial distance	Number of pursuers
Image–target pairs	Aim offset	Generation seed ξ

Driving template example

Route length limited to [200, 500] m, turns ∈ {2, 3, 4, 5}; together they define candidate routes, from which the generator builds the full route and initial state

Evasion template example

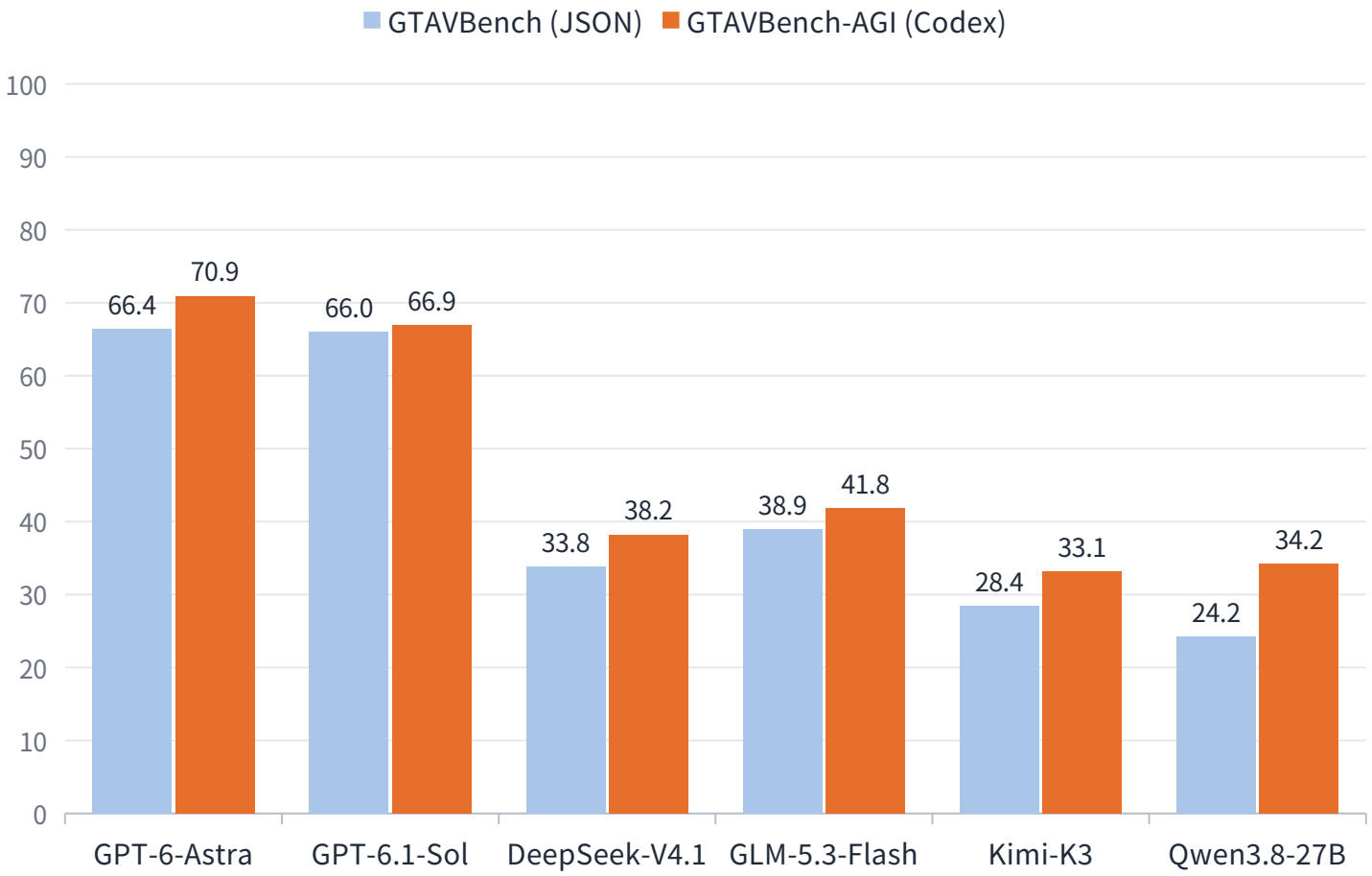
Pursuers ∈ {1, 2, 3}, initial distance [20, 40] m, speed [2, 4] m/s; seed ξ determines the concrete values

Sniping is used only for evaluation, not for environment generation

Source: paper Section 3.4, Eq. (1), Appendix B, Tables 7/8

GPT models far ahead: best 70.9%, yet 18.9 pt below humans

Overall accuracy (%, 150 tasks, mean of 3 seeds)



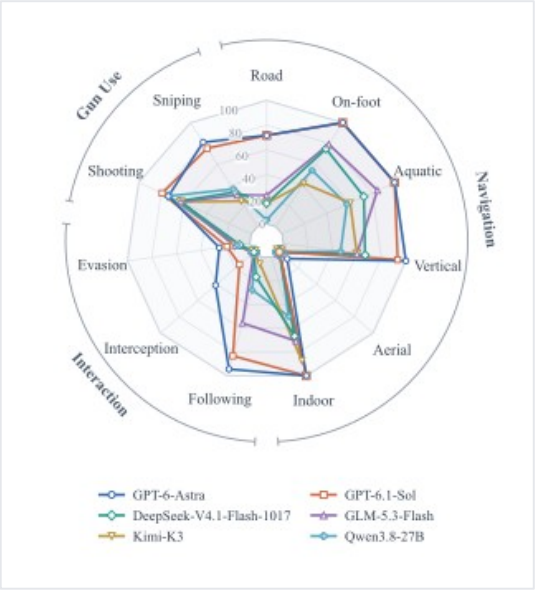


Fig. 1: 6 models' accuracy on 11 categories (Codex harness)

89.8%

Human avg.
(3 players)

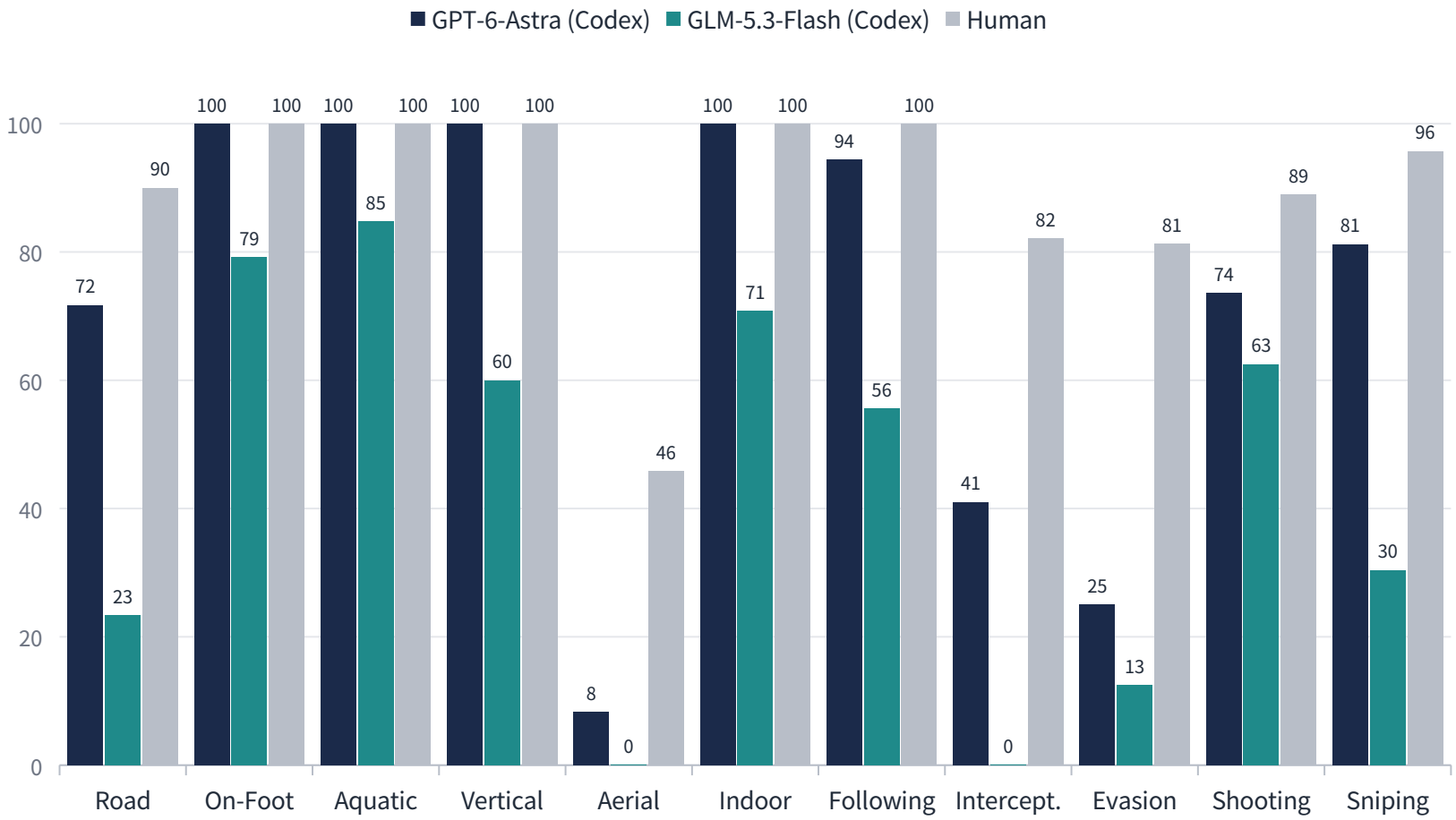
AGI protocol lifts all 6 models by **+0.9 to +10.0 pt**

Note: image instructions and harness changed together; not ablated separately

Data: paper Table 2; right: Figure 1

Static nav. near 100%; flight, interception, evasion nearly all fail

Per-category accuracy (%)



Aerial

⩽ 8.3%

All models 0–8.3%; humans only 45.8%

Interception

⩽ 41.0%

Only Astra + Codex reaches 41.0 ± 18.1; others 0–15.4%; humans 82.1%

Evasion

⩽ 25.0%

Models at most 25.0%; humans 81.3%

Data: paper Table 2 (GTAVBench-AGI and humans)

Efficiency

High accuracy ≠ more thinking: GPT uses only ~2–3k output tokens per task

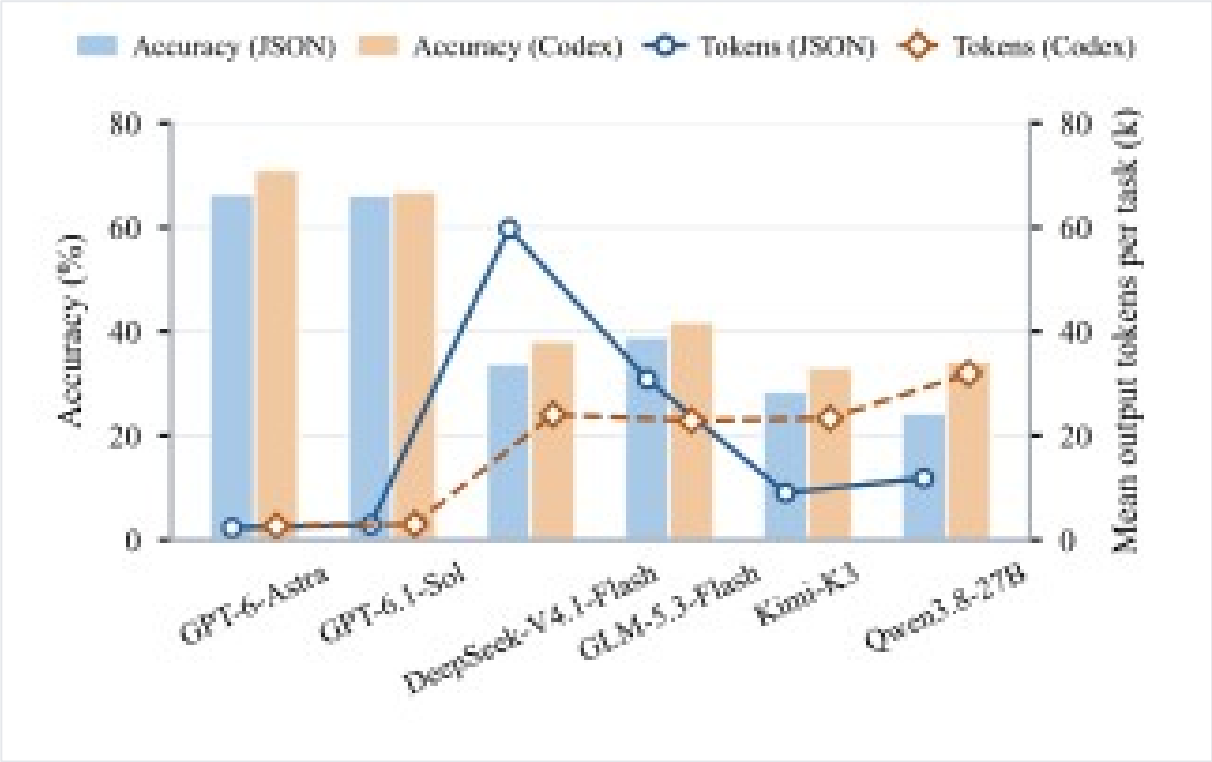


Figure 3: accuracy (bars) and mean output tokens per task (line)

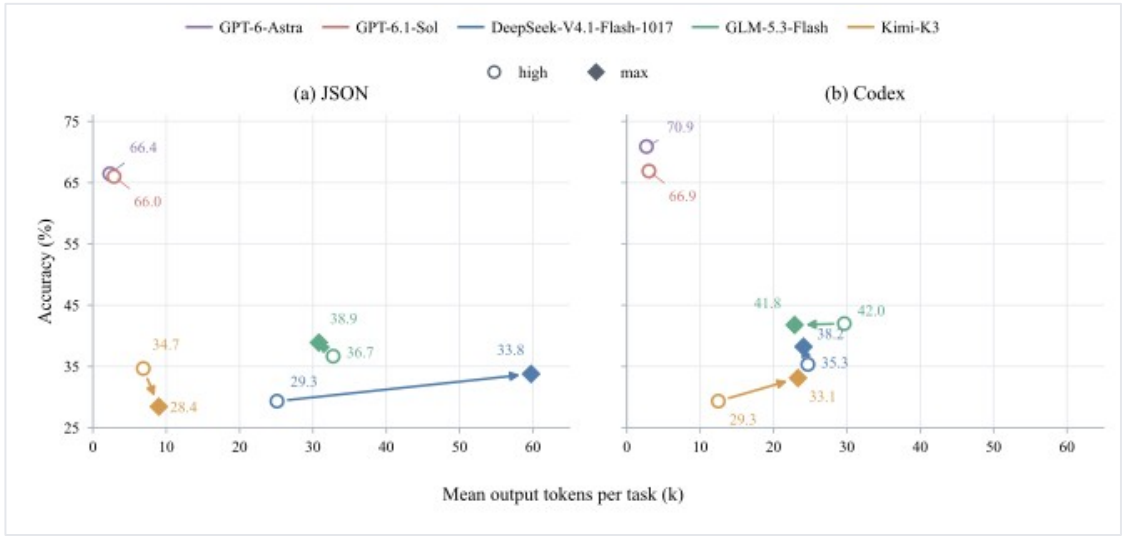


Figure 8: reasoning effort high (hollow) vs max (filled)

GPT models ~2–3k output tokens per task, yet the highest accuracy

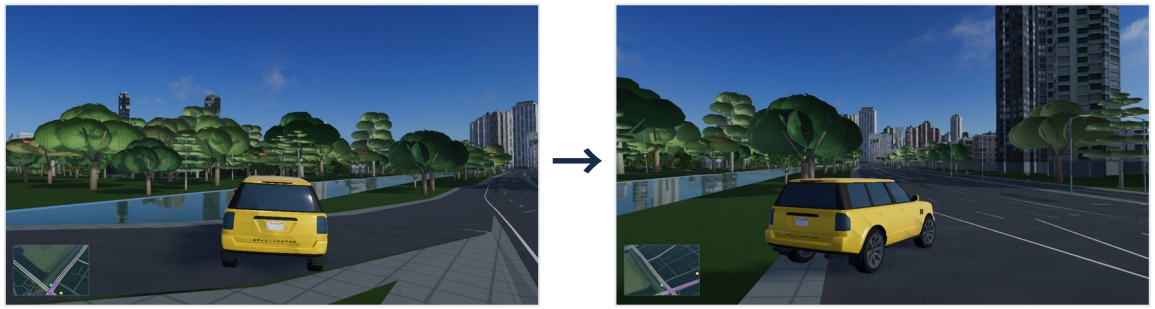
DeepSeek (JSON) ~26× Astra's output tokens, accuracy 33.8%

high → max unstable: Kimi JSON 34.7 → 28.4; DeepSeek JSON 29.3 → 33.8, but with far more tokens

Figures: paper Figure 3 (left), Figure 8 (right)

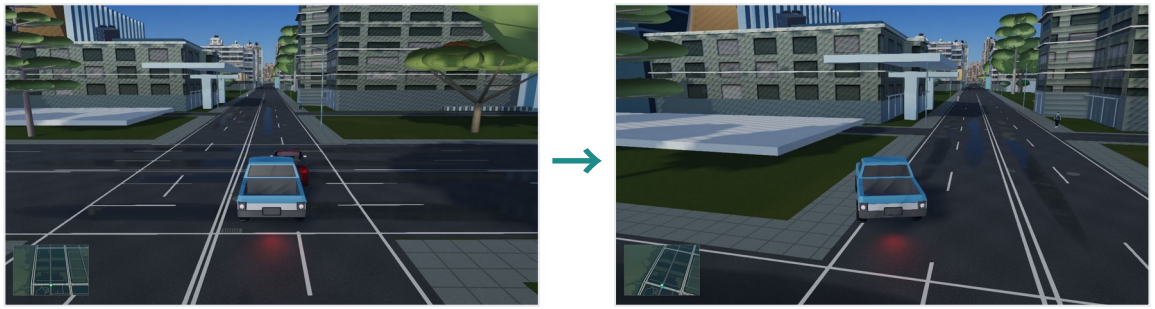
Four typical failures: the problem is control, not understanding

Road driving • Kimi-K3 (JSON) Cause: **oversteers and keeps accelerating**



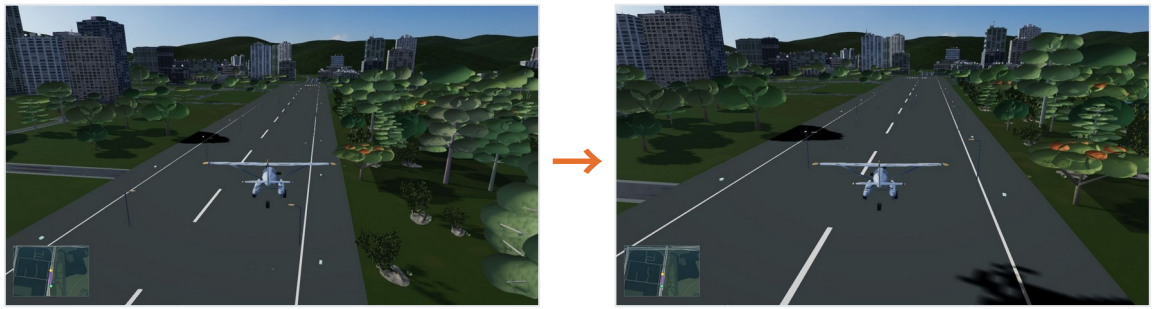
W 1 s + D 1 s: speed 24.2 → 30.5 m/s, turns ~67° onto the curb

Interception • GPT-6.1-Sol (Codex) Cause: **no braking after impact, block lost**



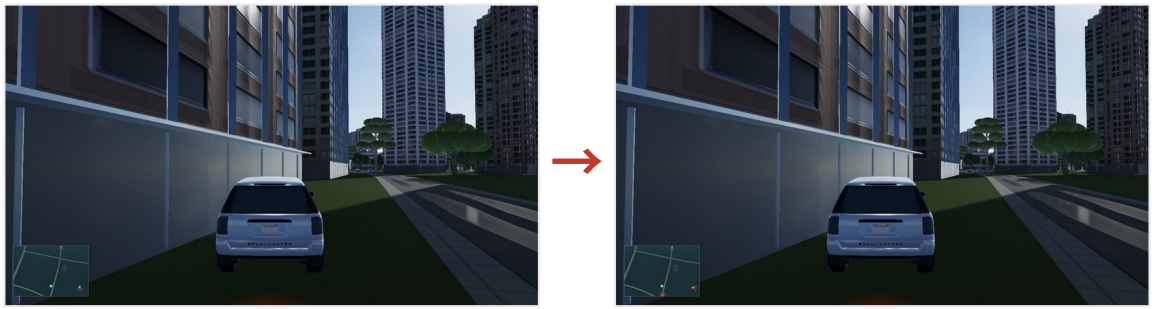
After impact the target drops to 0.2 m/s, agent still at 37.5 m/s, overshoots 20.4 m and loses contact

Aerial landing • GPT-6.1-Sol (Codex) Cause: **sink-rate correction too late**



S 1 s to cut throttle: altitude 18.6 → 9.4 m, still sinking at 9.2 m/s, then crashes

Evasion • GPT-6.1-Sol (Codex) Cause: **pinned to a wall, reverses too late**

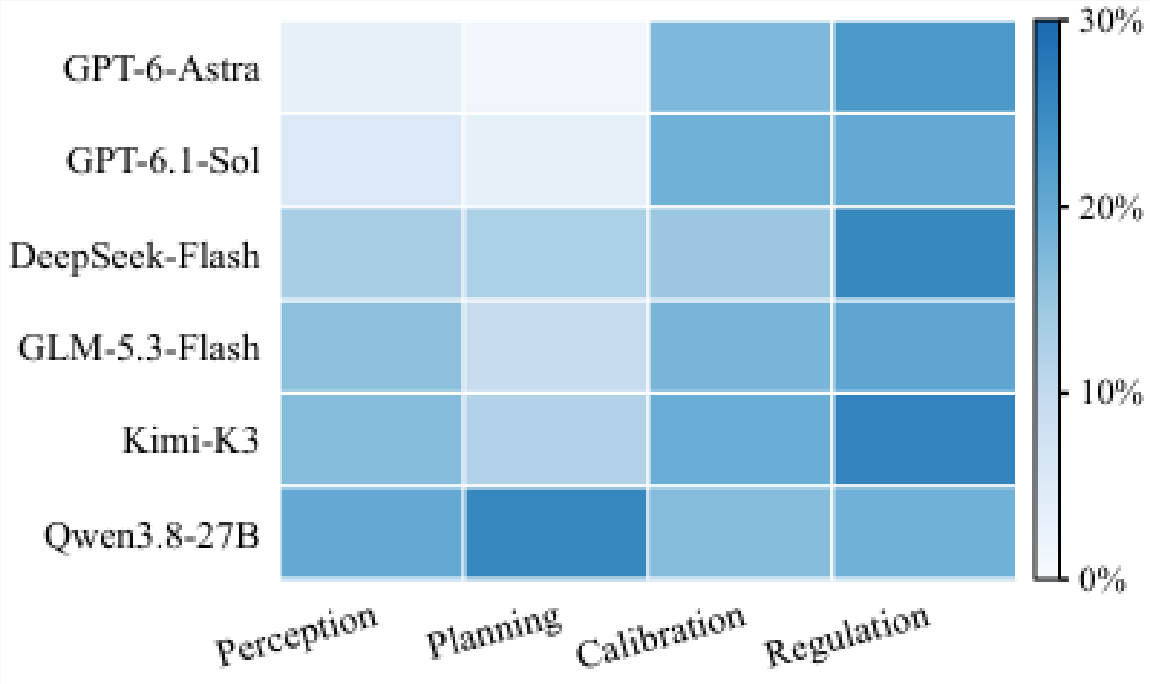


After hitting a building, keeps W 1 s + D 0.4 s: zero displacement for a full 1 s

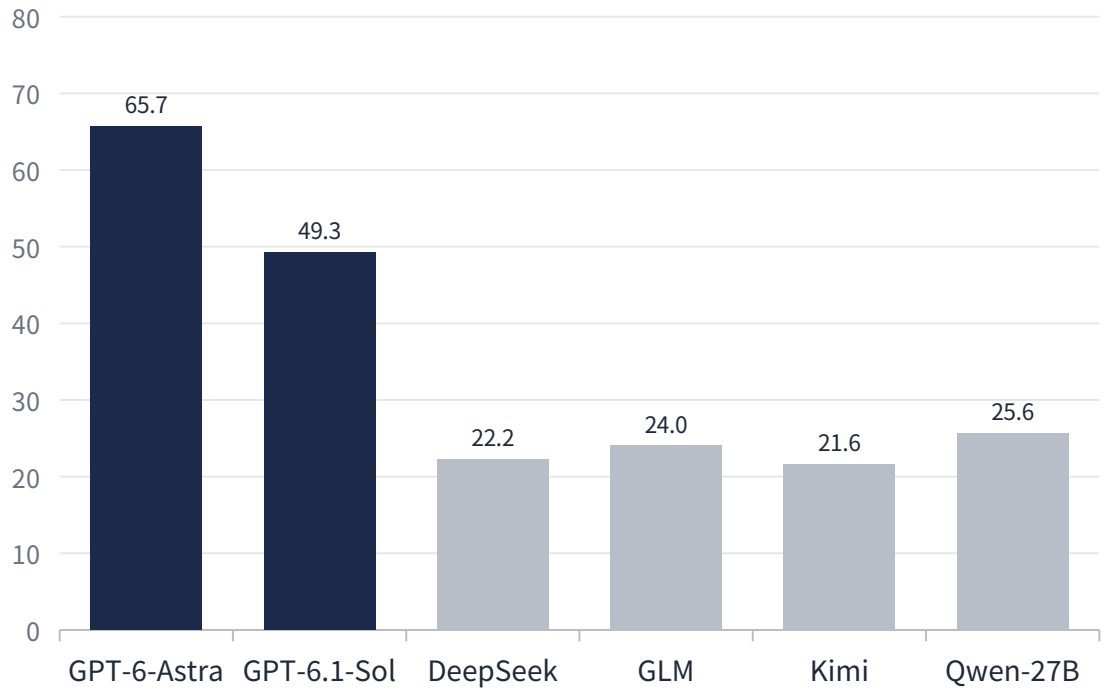
Figure: paper Table 11 (raw observations before → after action)

63.2% of first errors: calibration/regulation; GPT recovers better

First-error rate (each cell = first errors of that type / 150)



Recovery rate within 10 steps after first error (%)



112	96	157	200
Perception	Planning	Calibration	Regulation

Overall recovery 179 / 565 = **31.7%**; calibration + regulation account for 63.2% of the 565 first errors

Sample: 1 run per model under the JSON protocol, 900 trajectories in total; 565 have a locatable, classifiable first error

Figure: paper Figure 6 (left); data: Appendix C, Table 13

Long memory aids strategy switching; compaction may drop the goal

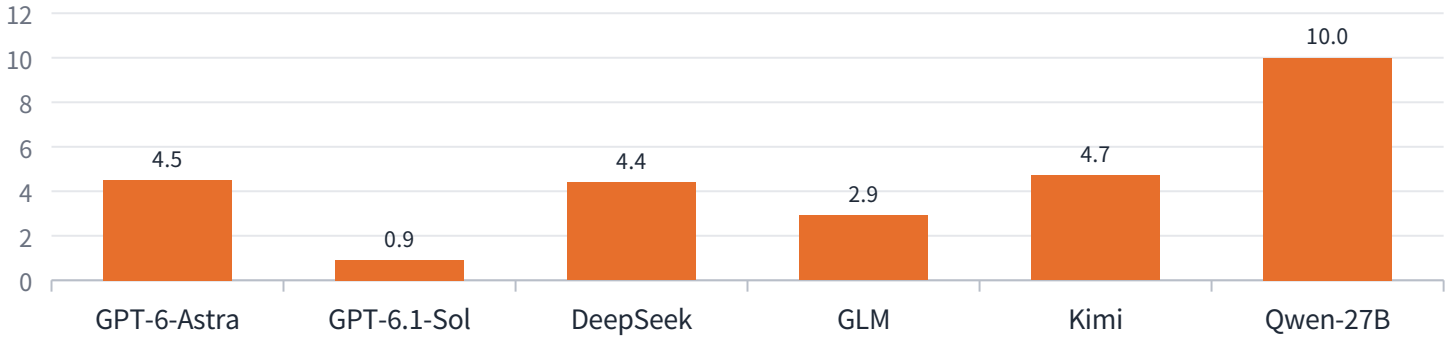
Helps: remembering tried strategies

- Keeps longer history than the JSON protocol's 10-image window, with text summaries for coherence
- GPT-6-Astra intercepting a motorcycle: recalls a blocked approach route and tries another maneuver
- DeepSeek-Flash indoor navigation: remembers searched rooms and failed door-opening attempts

Risk: errors persist, goals get replaced

- Long history and compaction can also preserve misunderstandings and weaken goal persistence
- Kimi-K3: correctly identifies the rooftop goal, switches to a driving goal after compaction, finally dropping the original goal from the summary
- GLM-5.3-Flash: carries an inverted reading of minimap markers into later decisions

Overall gain of AGI (Codex) over the JSON protocol (pt)



Takeaway

Context handoff (compaction) points are hotspots for trajectory drift and goal loss; harness design is itself part of agent capability.

Source: paper Section 5.3; data: Table 2

No inference pause: −22.3 pt accuracy; latency stalls dynamic interaction

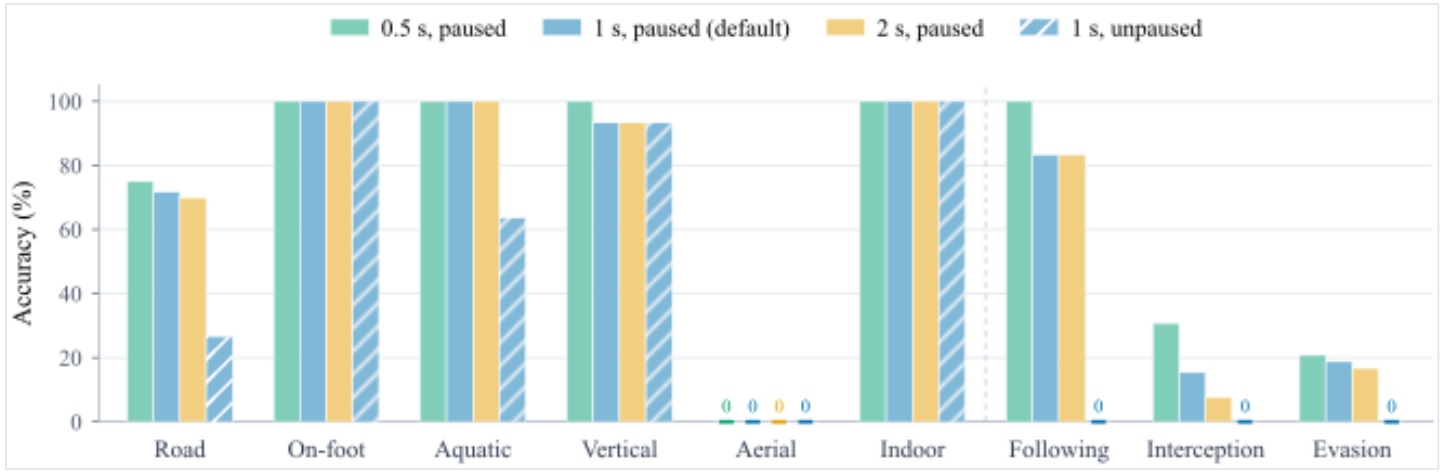


Figure 7: per-category accuracy under four key-duration cap / pause settings

d_max	Pause	3D acc.	Turns	Input tokens
0.5 s	Yes	66.3%	121.2	11.20M
1 s*	Yes	62.1%	62.6	4.32M
2 s	Yes	60.5%	42.5	2.50M
1 s	**No**	**39.8%**	**31.8**	**1.83M**

* Default setting; per-task means

- **0.5 s:** more frequent feedback; Following reaches 100%, Interception ~31%; cost: 1.9× turns, 2.6× input tokens
- **2 s cap:** 66.6% of actions exceed 1 s; models fail to choose suitable durations
- **No pause:** Following / Interception / Evasion all drop to zero; On-Foot and Indoor stay at 100%

Interpretation: main-table results assume the game waits for the model to think, which is optimistic for real-time deployment.

Figure: Figure 7; data: Section 5.4, Table 5 (GPT-6.1-Sol + Codex)

LoRA-GRPO on 50 generated on-foot tasks: +18.7 pt in-distribution

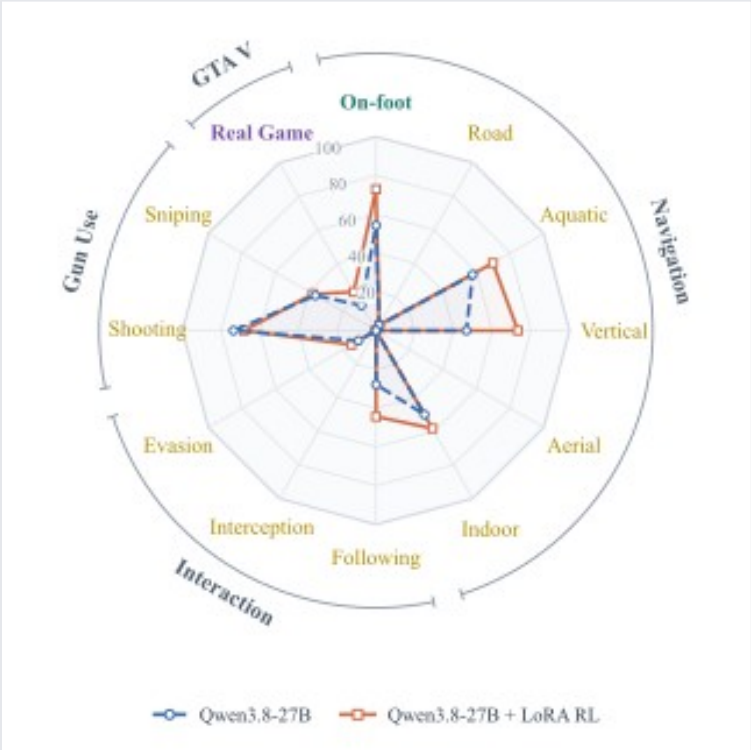
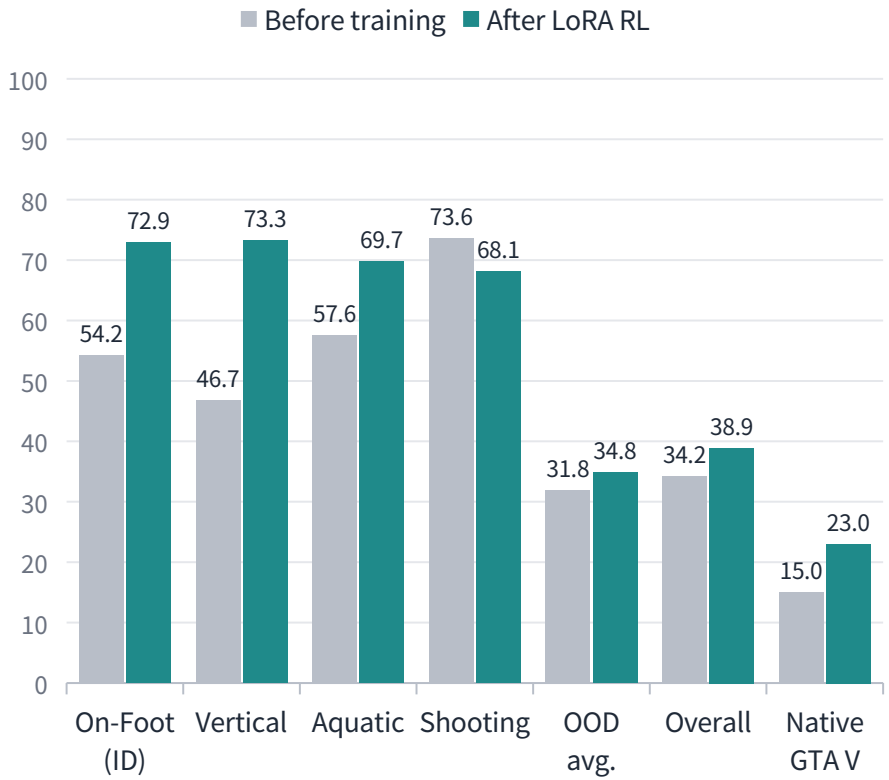


Fig. 4: green = ID, yellow = OOD, purple = native GTA V

Qwen3.8-27B accuracy before / after training (%)



Training setup

- 50 generated on-foot instances, routes spatially disjoint from the benchmark
- GRPO + LoRA: r = 32, α = 64 (q/v projections)
- 4 tasks × 8 rollouts per step; 8 × H200
- Trained inside the Codex harness

Terminal reward

$$R = S + 0.3(1-S)\cdot P - 0.1\cdot \mathbb{1}[N>100]$$

P: final progress by A* traversable distance; partial credit only for failed rollouts

No gains: Road still 3.3%, Aerial and Interception still 0, Shooting 73.6 → 68.1; Vertical has only 5 tasks, native GTA V is judged manually from recordings, and no significance is reported.

Figure: paper Figure 4; data: Section 4.3, Appendix B.1–B.3

Solid engineering, clear diagnosis; caveats: “real AAA”, sample size

Strengths

1. Complete engineering: detailed predicates (anti-tunneling, termination priority, K bullets for K targets), strict action validation, Web parallelism, reproducible seeds
2. Strong discrimination: static navigation near saturation, dynamic control almost entirely failing, clearly pinpointing weaknesses
3. Actionable diagnosis: first-error and recovery analyses point to calibration and regulation, not perception
4. Pause / duration ablations quantify the tension between “slow-thinking VLMs” and real-time control; the RL loop extends all the way to native GTA V

Limitations

1. “Real AAA” needs a caveat: 3D tasks run in a simplified rebuilt city, with routes merely substitutes of similar turn count
2. Small per-category sizes: Vertical 5, Following 6, Aerial 8 tasks; each task is worth 12–20 pt
3. Confounded AGI gains: image instructions and Codex context management changed together, not ablated separately
4. Main table relies on the inference pause, optimistic for real-time deployment
5. Small-scale RL: only the on-foot category is trained, with a host-side A* distance reward; no gains on Road / Aerial / Interception

Reviewer's assessment based on the paper

Summary

Three takeaways, and who should read this paper

01	**Verifiable + Generative**	GTAGym unifies AAA task structure, engine-state scoring and RL instance generation in one Web-parallel environment
02	**Bottleneck: closed-loop control**	Flight, interception, evasion and inference latency are the next frontiers; 63.2% of first errors are calibration or regulation
03	**Small-scale RL already helps**	50 generated tasks give +18.7 pt in-distribution, with partial transfer to unseen categories and native GTA V (15% → 23%)

Who should read

Game / computer-use agent evaluation

Ready-to-use, state-scored AAA-style benchmark + parallel Web interface

Agentic RL

Parameterized environment generator, sparse-reward interface and a reference GRPO + LoRA config

Real-time embodied control

Pause / duration ablations and error distributions point to two directions: inference latency and continuous-control precision

arXiv 2611.13028 • Code: github.com/ljbro/GTAGym